

MERN STACK MACHINE TEST

Task Title: EduSphere — Multi-Tenant Student Information & Learning Management System

Duration: 3 Hours Total Marks: 100

Overview: Build a production-ready Student Information System and Learning Management Platform where multiple educational institutions can manage students, teachers, courses, attendance, examinations, and role-based access from a single platform.

Supported Roles: Super Admin, Institute Admin, Teacher, Student **Stack:** React.js / Next.js, Node.js + Express.js, MongoDB or PostgreSQL, JWT.

1. Backend — 40 Marks

Method	Route	Access	Requirement
POST	/api/auth/register	Public	Register user
POST	/api/auth/login	Public	Login user
POST	/api/auth/refresh-token	Public	Generate new access token
GET	/api/auth/me	Authenticated	Get current user

- Password hashing using bcrypt
- JWT access token and refresh token
- Protected routes
- Role-based authorization
- Validation and error handling

2. Student Management — 20 Marks

Fields: firstName, lastName, email, phone, dateOfBirth, gender, course, institute, profileImage, admissionNumber, status

APIs: POST /api/students, GET /api/students, GET /api/students/:id, PUT /api/students/:id, DELETE /api/students/:id

- Create, update, and delete student
- Search/filter by course or status
- Pagination
- Profile image upload
- Institute-level data isolation

3. Course & LMS Module — 15 Marks

Course: title, description, category, instructor, thumbnail, lessons, status **Lesson:** title, videoUrl, description, duration, order

- Teacher can create courses and add lessons
- Students can view enrolled courses
- Track course progress
- Mark lessons as completed

4. Attendance Module — 10 Marks

Fields: studentId, courseId, date, status (Present / Absent / Leave) **APIs:** POST /api/attendance, GET /api/attendance/student/:studentId

Dashboard: Total Classes, Present, Absent, Leave, Attendance Percentage

5. Frontend — 25 Marks

- Login with JWT token handling and Register with validation
- Role-based dashboard, protected routes, and role-based routes
- Student management table/form and Course list/details
- Attendance summary
- Search, pagination, loading, error, success, and empty states

Dashboards: Super Admin — Institutes, Students, Teachers, Courses | Institute Admin — Students, Teachers, Courses, Attendance | Teacher — My Courses, My Students, Attendance | Student — My Courses, Attendance, Results, Learning Progress

6. Image Upload — 5 Marks

- Support student profile image and course thumbnail
- Store image URL/path in database
- Display image on frontend
- Support image update

7. Code Quality & Documentation — 10 Marks

Backend: config, controllers, models, routes, middleware, services, utils, app.js, server.js, .env.example, README.md

Frontend: components, pages, layouts, services, hooks, context, utils, App.jsx

- Clean modular code
- Proper validation and error handling
- Environment variables
- API documentation
- Setup instructions

Bonus / Optional Advanced Features

- **Socket.IO:** Real-time announcements, notifications, live attendance updates, and optional teacher-student messaging.
- **Docker:** Containerize frontend, backend, and database services for consistent development and deployment.
- **Docker Compose:** Use docker-compose.yml to configure and run the complete multi-container application with a single command.
- **Redis:** Caching for student lists and dashboard statistics.
- **Background Jobs:** Asynchronous email notifications and scheduled reminders.

Deployment & Submission Requirements

- **Deployment:** Deploy the frontend and backend application to a publicly accessible hosting/server environment. The submitted project should include working deployment URLs where applicable.
- **Source Code:** Submit the complete working project with clean modular code, .env.example, and README.md containing setup instructions and API details.
- **Repository:** Push the complete project to a public GitHub repository and provide the repository link.
- **Documentation:** README.md should include local setup, environment variables, API documentation, deployment instructions, and any required Docker/Docker Compose commands.

Evaluation Area

Area	Marks
Authentication & JWT	15
Student Management APIs	20
Course & LMS Module	15
Attendance Module	10
Frontend Dashboard & UI	20
Image Upload	5
Code Quality & Documentation	15
TOTAL	100